# KOLOKWIUM 4

Algorytmy na tekstach

**UWAGA:** Odpowiedzi do każdego zadania należy przesłać w formie poszczególnych plików (**jeden plik na jedno zadanie**). Całość rozwiązań powinna zostać spakowana do jednego archiwum (np. w formacie **.zip** lub **.tar.gz**) przed wysłaniem.

## Zadanie 1

Dokonano zaszyfrowania tekstów jawnych przez szyfr Cezara. Uzupełnij tabelę wiedząc, że korzystamy wyłącznie z podstawowych liter alfabetu łacińskiego (26 znaków) bez polskich znaków.

| **Tekst jawny** | **Przesunięcie (dodatnie)** | **Tekst niejawny** |
|---|---|---|
| MONKEY | 4 | |
| | 7 | ABELZ |
| ZADKY | 12 | |
| | | RABAN |

### Wskazówka

Kolejne litery alfabetu to ABCDEFGHIJKLMNOPQRSTUVWXYZ.

## Zadanie 2

Poniższa tabela to wycinek klucza kodującego w szyfrowaniu przez podstawienie. Uzupełnij ją dla podanych znaków tak, aby szyfrowanie za jej pomocą dawało identyczne rezultaty, jak szyfr Cezara z przesunięciem o 5.

| **Znak jawny** | A | H | R | | I | Z | B | | W | L |
|---|---|---|---|---|---|---|---|---|---|---|
| **Znak niejawny** | | | | U | | | | H | | |

## Zadanie 3

Dany jest ciąg znaków $S$. Zaimplementuj w języku **Python** funkcję `can_form_palindrome(S)`, która sprawdzi, czy istnieje taki **anagram** tego ciągu, który jest **palindromem**. Funkcja powinna zwracać wartość `True`, jeśli ułożenie palindromu jest możliwe, oraz `False` w przeciwnym przypadku.

### Przykład 1

Dla danych:

```
S = aabbc
```

Wywołanie `can_form_palindrome(S)` powinno zwrócić wynik `True`. Przykładowy anagram będący palindromem to `"abcba"`.

### Przykład 2

Dla danych:

```
S = abbaccd
```

Wywołanie `can_form_palindrome(S)` powinno zwrócić wynik `True`. Przykładowy anagram będący palindromem to `"abcdcba"`.

### Przykład 3

Dla danych:

```
S = python
```

Wywołanie `can_form_palindrome(S)` powinno zwrócić wynik `False`.

### Przykład 4

Dla danych:

```
S = racecar
```

Wywołanie `can_form_palindrome(S)` powinno zwrócić wynik `True`. Ciąg już jest palindromem, więc warunek jest spełniony.

## Zadanie 4

**Ukośny szyfr przestawieniowy** (ang. *slanted transposition cipher*) koduje **tekst jawny** za pomocą macierzy o ustalonej liczbie wierszy $r$.

Znaki są wpisywane do macierzy *po przekątnej*, zaczynając od lewego górnego rogu i kierując się w prawo-dół. Gdy jedna przekątna zostanie wypełniona, kolejna zaczyna się od górnej komórki następnej wolnej kolumny. Puste komórki uzupełniane są spacją ␣. **Liczba kolumn** $c$ **dobierana jest tak, by ostatnia kolumna *nie* była pusta**. **Tekst szyfrowany** powstaje przez odczytanie macierzy *wierszami*, od lewej do prawej.

### Przykład

**Tekst jawny:** `"szyfry"`, liczba wierszy $r = 3$. Znaki wpisujemy po przekątnych:

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | s | f | | |
| 1 | | z | r | |
| 2 | | | y | y |

Odczytując macierz wierszami (spacje oznaczono ␣):

$$\underbrace{\texttt{s f _ _}}_{\text{Wiersz 0}} \;\Big|\; \underbrace{\texttt{_ z r _}}_{\text{Wiersz 1}} \;\Big|\; \underbrace{\texttt{_ _ y y}}_{\text{Wiersz 2}} \implies \texttt{tekst szyfrowany} = \texttt{sf␣␣␣zr␣␣␣yy}$$

### a)

Uzupełnij poniższe macierze oraz podaj **tekst szyfrowany** (użyj znaku ␣ dla spacji).

1. **Tekst jawny:** `BIT`, liczba wierszy $r = 2$.

| | |
|---|---|
| | |
| | |

**Tekst szyfrowany:** .....

2. **Tekst jawny:** `KOD`, liczba wierszy $r = 3$.

| | | |
|---|---|---|
| | | |
| | | |
| | | |

**Tekst szyfrowany:** .....

### b)

Odczytaj **tekst jawny** na podstawie podanych danych. Zwróć uwagę na spacje wewnątrz tekstu oraz fakt, że **tekst jawny** nie może kończyć się spacjami (*trailing spaces*).

1. **Tekst szyfrowany:** `"H␣␣␣E␣␣␣L␣␣␣L␣␣␣O␣␣␣"`, $r = 4$
   **Tekst jawny:** .....
2. **Tekst szyfrowany:** `"b␣i␣t␣␣m␣a␣t"`, $r = 2$
   **Tekst jawny:** .....
3. **Tekst szyfrowany:** `"algorytmy"`, $r = 1$
   **Tekst jawny:** .....

### c)

Wyznacz zależność między długością **tekstu szyfrowanego** $n$, liczbą wierszy $r$ oraz liczbą kolumn $c$ macierzy.

### d)

Napisz w języku **Python** funkcję `decode(encoded_text, rows)`, która przyjmuje **tekst szyfrowany** oraz liczbę wierszy $r$, a następnie zwraca zdekodowany **tekst jawny** (bez spacji końcowych).

*Powodzenia!*